Part 1 — Requirements & UML (10 pts)

What MentorBridge Does

MentorBridge is a website where college students can find alumni mentors, schedule meetings with them, and leave reviews after. Mentors can see their schedule and past sessions. School staff can see how much the platform is being used.

The main pieces of data we store:

- Students (name, email, major, graduation year)
- Mentors (name, email, company, job title, years of experience)
- Sessions (which student met which mentor, when, and about what)
- Feedback (rating and comment left after a session)
- Industries (like Tech, Finance, etc. Both students and mentors can be linked to multiple)

Relationships:

- One student can have many sessions
- One mentor can have many sessions
- Each session can have one feedback
- Students and mentors can both be linked to multiple industries

What we're using Redis for:

1. Session Cart — when a student is browsing mentors and picking who they want to meet, we hold that list in Redis temporarily instead of saving it to the database right away
2. Top Mentors Leaderboard: we keep a live ranking of the highest-rated mentors in Redis so the homepage loads fast without hitting the database every time
3. Who's Currently Logged In: we store active login tokens in Redis so we can quickly check if someone is logged in and show staff how many users are online

Part 2 — Redis Data Structures (30 pts)

Session Cart → Redis Hash

For the session cart I'll use a Redis Hash with the key cart:<student_id> (e.g. cart:42). Each item in the cart is stored as a field like entry:1 and the value is a small JSON object with the mentor, date, and topic. A Hash works well here because all of a student's cart items live under one key, you can add or delete individual items easily, and reading the whole cart or just one item is fast. The cart automatically deletes itself after 24 hours if the student never checks out.

Top Mentors Leaderboard → Redis Sorted Set

For the leaderboard I'll use a Redis Sorted Set with the key leaderboard:mentors. Each mentor's ID is stored as a member and their average rating (multiplied by 100 to keep it as a whole number, so 4.75 becomes 475) is their score. Every time someone leaves feedback, that mentor's score gets updated. To show the top 5 on the homepage we just ask Redis for the top 5 no complex database query needed. This stays in Redis permanently and just gets updated as new reviews come in.

Active Logged-In Users → Redis Set

For tracking who's logged in I'll use a Redis Set with the key active:users. When someone logs in, their session token (a unique ID) gets added to the set. There's also a separate key session:<token> that stores which user that token belongs to, and it expires after 1 hour

automatically. To check if someone is still logged in we just ask Redis if their token is in the set very fast. Staff can see how many people are online at any moment just by asking for the size of the set.

Part 3 — Redis Commands (30 pts)

Session Cart (Hash)

- Start a new cart for student 42 when they begin browsing: HSET cart:42 entry:1 '{"mentor_id":7,"date":"2026-05-01","topic":"Resume Review"}'
- Set the cart to disappear after 24 hours if they never finish: EXPIRE cart:42 86400
- Show everything currently in the cart: HGETALL cart:42
- Look at one specific item in the cart: HGET cart:42 entry:1
- Student changed their mind and wants a different date: HSET cart:42 entry:1 '{"mentor_id":7,"date":"2026-06-01","topic":"Resume Review"}'
- Student removed one mentor from their cart: HDEL cart:42 entry:1
- Show how many mentors are in the cart right now: HLEN cart:42
- Student submitted everything so we delete the cart: DEL cart:42

Top Mentors Leaderboard (Sorted Set)

- Add mentor 7 to the leaderboard for the first time with a score of 475 (which represents a 4.75 average rating): ZADD leaderboard:mentors 475 "7"
- Show the top 5 mentors on the homepage: ZREVRANGE leaderboard:mentors 0 4 WITHSCORES
- Show every mentor on the leaderboard: ZREVRANGE leaderboard:mentors 0 -1 WITHSCORES
- A new review came in so we update mentor 7's score: ZADD leaderboard:mentors 485 "7"
- Bump mentor 7's score up by 10 points: ZINCRBY leaderboard:mentors 10 "7"
- See what place mentor 7 is currently in: ZREVRANK leaderboard:mentors "7"
- See mentor 7's actual score: ZSCORE leaderboard:mentors "7"
- Mentor left the platform so we remove them from the leaderboard: ZREM leaderboard:mentors "7"

Active Logged-In Users (Set)

- Clear everything out when the server restarts: FLUSHALL
- Student logs in so we save their session token: SADD active:users "a3f9c2d1-84b0-4e6f-9abc-123456789abc"
- Save which user that token actually belongs to: SET session:a3f9c2d1-84b0-4e6f-9abc-123456789abc 42
- Set that token to expire automatically after 1 hour: EXPIRE session:a3f9c2d1-84b0-4e6f-9abc-123456789abc 3600
- Check if a user is still logged in before letting them do something: SISMEMBER active:users "a3f9c2d1-84b0-4e6f-9abc-123456789abc"
- Figure out which student this token belongs to: GET session:a3f9c2d1-84b0-4e6f-9abc-123456789abc

- Staff wants to see how many people are online right now: SCARD active:users
- Staff wants to see all active tokens: SMEMBERS active:users
- User is still active so we reset their 1 hour timer: EXPIRE session:a3f9c2d1-84b0-4e6f-9abc-123456789abc 3600
- User clicks log out so we remove their token: SREM active:users "a3f9c2d1-84b0-4e6f-9abc-123456789abc"
- Delete the session key too since they logged out: DEL session:a3f9c2d1-84b0-4e6f-9abc-123456789abc

UML Diagram Code

```
classDiagram
    direction LR

    class Student {
        <<Entity>>
        +int student_id PK
        +string first_name
        +string last_name
        +string email
        +string major
        +int graduation_year
    }

    class Mentor {
        <<Entity>>
        +int mentor_id PK
        +string first_name
        +string last_name
        +string email
        +string company
        +string job_title
        +int years_experience
    }

    class Session {
        <<Entity>>
        +int session_id PK
        +int student_id FK
        +int mentor_id FK
        +date session_date
        +int duration_minutes
        +string status
        +string topic
```

```
}

class Feedback {
    <<Entity>>
    +int feedback_id PK
    +int session_id FK
    +int rating
    +string comment
    +datetime submitted_at
}

class Industry {
    <<Entity>>
    +int industry_id PK
    +string industry_name
    +string description
}

class SessionCart {
    <<Redis Hash>>
    +string key: cart~student_id~
    +string field: entry~n~
    +string value: mentor_id, date, topic
    +int TTL: 86400 seconds
}

class TopMentorsLeaderboard {
    <<Redis Sorted Set>>
    +string key: leaderboard~mentors~
    +string member: mentor_id
    +int score: avg_rating x 100
}

class ActiveUsers {
    <<Redis Set>>
    +string key: active~users~
    +string member: session_token
    +string companion: session~token~ = user_id
    +int TTL: 3600 seconds
}

Student "1" --> "0..*" Session : books
Mentor "1" --> "0..*" Session : conducts
Session "1" --> "0..1" Feedback : receives
```

Student "0..*" --> "0..*" Industry : is interested in
Mentor "0..*" --> "0..*" Industry : works in
Student "1" --> "1" SessionCart : has cart
Session "1" --> "1" TopMentorsLeaderboard : updates score
Student "1" --> "1" ActiveUsers : login tracked